

Claude Code usage | Best practices for cost optimization and workflow efficiency

[Project IT Environment](#) | [AI API Products Management](#)

This page provides the information on how to use Claude Code efficiently, reduce unnecessary generation costs, maintain output quality, and manage session context effectively. It is applicable to developers and other SoftServe associates who need to use Claude Code.

Table of Contents:

- [Why efficient usage matters](#)
- [Recommended setup](#)
- [Best practices for cost, quality, and efficiency](#)
- [How to understand Claude Code usage cost](#)
- [How to use Claude Code insights](#)
- [Recommended workflow](#)
- [Key commands](#)

Why efficient usage matters

Claude Code generation costs can vary significantly between sessions, even when the output quality is similar. A well-managed session can cost 5–10 times less than a poorly managed one.

The main reason is context management. Long sessions, repeated retries, and oversized contexts increase token usage and reduce output quality.

Recommended setup

Use the following setup as the default configuration:

- **Model:** Sonnet 4.6. Use Sonnet 4.6 / sonnet as the recommended default for most coding tasks but verify the active model with `/status` or `/model`, and pin it via managed settings if needed. For more details, follow the [Model configuration - Claude Code Docs](#) vendor documentation.
- **Context window:** 200k.

- **Extended context:** Disabled unless required.
- **Effort:** Medium.
- **Skill selection:** Choose the proper skill for each task.
- **MCP:** Connect Figma MCP when working with design-related tasks.



Important

Use higher reasoning models, such as Opus, only when the task clearly requires advanced reasoning or deeper analysis.

Best practices for cost, quality, and efficiency

Use the following best practices to improve output quality, reduce unnecessary token usage, and manage Claude Code sessions more efficiently. These recommendations help you maintain focused context, avoid excessive costs, and achieve more consistent results.

DO	DON'T
Use Sonnet 4.6 as the default model. Switch models only with a clear reason.	Do not use high reasoning models, such as Opus, without a clear need. Higher model complexity does not automatically improve results.
Read summaries at every step and clarify issues before continuing.	Do not continue working when summaries are unclear or incorrect.
Clear the context between tasks to maintain focus and reduce costs.	Do not let context usage grow excessively during small fixes or unrelated tasks.
Break large features into smaller focused tasks.	Do not combine many unrelated systems or 10–15+ files into one session.
Batch 3–5 related fixes or components into one prompt.	Do not create separate prompts for every small change.
Monitor token usage regularly with <code>/cost</code> .	Do not ignore cache growth or <code>/context</code> warnings.
Rephrase the problem or start fresh after 2–3 failed attempts.	Do not repeat many retries for the same issue in one session.
Keep prompts focused and specific.	Do not use broad requests such as “refactor everything.”

DO	DON'T
Use <code>/clear</code> when switching tasks.	Do not continue unrelated work in the same context.
Balance task decomposition with efficiency.	Do not over-decompose tasks into many tiny prompts.
Provide verification criteria: tests, expected output, screenshots, lint/type check commands.	Do not accept “done” without verification for production code.
Use <code>/plan</code> for ambiguous, multi-file, or architecture-heavy changes.	Do not jump directly into implementation when the approach is unclear.
Use <code>Esc</code> , <code>/rewind</code> , or “undo that” as soon as Claude goes in the wrong direction.	Do not let a wrong approach continue and then try to fix everything later.
Keep <code>CLAUDE.md</code> short and project-wide; move specialized workflows into Skills.	Do not overload <code>CLAUDE.md</code> with long docs, tutorials, or file-by-file descriptions.
Disable unused MCP servers and connect only the MCPs needed for the task.	Do not keep all MCPs enabled by default for every session.
Use subagents for broad investigation, review, or noisy log/test analysis.	Do not let the main session read hundreds of files or huge logs unless necessary.
Use <code>/rename</code> before <code>/clear</code> when the session may be useful later.	Do not lose useful work history just because you need a clean context.

For more details, follow the pages:

- [Best practices for Claude Code - Claude Code Docs](#)
- [Manage costs effectively - Claude Code Docs](#).

How to understand Claude Code usage cost

To understand usage cost, you should pay attention to such token types: **input**, **output**, **cache write**, **cache read**. Output is the most expensive (up to 5x input price). Cache read is cheap per **token** but adds up at **scale**.

In this example: 177.7k output on Opus = heavy generation, likely rework. 16.7M cache read = long session without clearing. 901k cache write = large context created each turn.

Rule: Run the command: `/cost` regularly. If output or cache numbers keep increasing—clear context and restart.

```

> /cost
└─ Total cost:          $19.55
   Total duration (API): 1h 5m 47s
   Total duration (wall): 2d 20h 43m
   Total code changes:  7850 lines added, 225 lines removed
   Usage by model:
     claude-sonnet-4-6: 34 input, 7.2k output, 868.0k cache read, 56.8k cache write ($0.58)
     claude-haiku-4-5: 774 input, 12.5k output, 630.7k cache read, 96.0k cache write ($0.2465)
     claude-opus-4-6: 58.5k input, 177.7k output, 16.7m cache read, 901.1k cache write ($18.72)

```

How to use Claude Code insights

You can run the command: `/insights` to analyze your last 30 days of Claude Code sessions.

It shows:

- Your projects
- Your interaction patterns
- Friction points
- Recommendations for improving efficiency.

How to use it: Run monthly. Focus on friction points—these are your **biggest cost** and **quality leaks**.

Recommended workflow

Before starting a task:

- 1 Choose the proper skill.
- 2 Select the default recommended model.
- 3 Estimate the task scope.
- 4 Decide whether the task should be split into smaller parts.

During the task

- 1 Read summaries carefully.
- 2 Monitor context size regularly.
- 3 Batch related small fixes together.
- 4 Run `/cost` during longer sessions.
- 5 Avoid unnecessary retries.

After completing the task

- 1 Review generation costs using `/cost`.
- 2 Clear the context before starting the next task.
- 3 Start the next feature in a fresh session when possible.

When stuck on a problem

Maximum recommended retries—2–3 attempts.

If the issue remains unresolved:

- Rephrase the request.
- Simplify the problem description.
- Start a fresh context.
- Switch models only if necessary.

Key commands

`/cost`

Shows token usage breakdown, including:

- Input tokens
- Output tokens
- Cache write tokens
- Cache read tokens.

Use this command to identify:

- Growing cache usage.
- Expensive generation patterns.

- Long inefficient sessions.

/insights

Provides recommendations based on your recent Claude Code usage. Generates a report analyzing Claude Code sessions, including project areas, interaction patterns, and friction points.

Use this command to:

- Identify workflow inefficiencies.
- Improve prompting habits.
- Reduce unnecessary costs.

/context

Shows what is currently loaded into the context window and where context is being spent. Use it before deciding whether to `/compact` or `/clear`.



Important

High context usage can reduce output quality before the context limit is reached.

/model

Switches models during the session.

Recommended usage:

- Start with Sonnet 4.6.
- Switch to Opus only when required.



When to use Opus?

Use Opus or higher effort only for tasks that require deeper reasoning: architecture decisions, complex debugging, security-sensitive changes, multi-step refactors, or cases where Sonnet failed after 2–3 well-scoped attempts. For routine implementation, small fixes, tests, documentation, and straightforward refactors, use Sonnet.

/usage

Shows session cost, plan usage limits, and activity stats. `/cost` and `/stats` are aliases. Use this regularly during longer sessions.

`/compact [instructions]`

Summarizes the current conversation to free up context while preserving important information. Use focused instructions, e.g. `/compact Focus on modified files, test results, and remaining TODOs.`

`/clear [name]`

Starts a fresh context for a new task. Use it when switching to unrelated work. Optionally name the previous session before clearing so it can be resumed later.